A Reproducible Benchmark of Relational Database Access-Layer Performance in Express.js: A Configuration-Specific Comparison on PostgreSQL and MySQL

Mateusz Miotk
Faculty of Mathematics, Physics and Informatics,
University of Gdańsk
mateusz.miotk@ug.edu.pl

July 15, 2026

Abstract

Context: Node.js/Express services reach relational databases through a spectrum of access layers (native drivers, query builders, and object-relational mappers, or ORMs), yet practitioners choose between them largely on vendor benchmarks that are fragmented, PostgreSQL-only, and rarely report the tail (p99). **Objective:** We present a reproducible, byte-equivalence-checked benchmark harness and use it to compare eleven access-layer implementations under one clearly specified operating point on PostgreSQL and MySQL; the harness and its controls, not a generalized library ranking, are the contribution. **Methods:** An identical Express application is served through nine idiomatic access layers (native drivers `pg` and `mysql2`, the query builder Knex, and the ORMs Drizzle, Prisma, Sequelize, TypeORM, Objection and MikroORM) and two tuned native baselines, over five access patterns. We report throughput and saturated client-observed response-time p99 jointly from 25 repeated runs per cell at a 50-connection operating point, complemented by a coordinated-omission-corrected open-loop tail measured at matched utilization; an automated cross-check verifies byte-identical responses across layers before measurement, and inserts run under the engines'

default durability. **Results:** For the benchmarked implementations, abstraction cost concentrates in relation-materializing patterns: the deep/nested fetch spreads 6.5× (PostgreSQL) and 4.5× (MySQL) between the native driver and the slowest data-mapper ORM. A same-SQL control narrows that native-relative spread to at most 1.5×; the residual is small, but the control changes query formulation, protocol, statement preparation, round-trips, and mapping together and isolates no single factor. Read rankings transfer across engines (Spearman $\rho \geq 0.89$); the write ranking does not ($\rho = 0.43$). At the saturated operating point Prisma matches or leads native throughput on several patterns, at roughly five times the application CPU of the others; at matched sub-saturation utilization the tail differences largely close. **Conclusion:** In this configuration the access layer matters most where it must assemble a nested object graph, but the durable contribution is the reproducible harness and a checklist of benchmarking pitfalls specific to this genre, released as a replication package.

Keywords: database access layer; object-relational mapping; Node.js; reproducible benchmarking; response-time tail (p99); empirical software engineering

1 Introduction

Express.js remains the most widely used web framework for Node.js, and almost every non-trivial service built on it eventually reaches a relational database through some *access layer*. The choice is not settled by the language or the framework: access layers trade raw performance for ergonomics, compile-time type safety, and protection from pitfalls such as the N+1 query problem, where a naive nested fetch silently issues one query per parent row instead of one for the whole result graph [9], a pervasive anti-pattern in real-world database-backed applications (Section 2). Native drivers expose the wire protocol directly (`pg` for PostgreSQL, `mysql2` for MySQL); query builders assemble SQL programmatically (Knex); and object-relational mappers (ORMs) map rows to objects and manage relationships automatically, following patterns such as Data Mapper and Active Record [21]. Node.js back-end performance under load is itself a recognized empirical topic [6], [7], [34], but that literature treats the runtime and web framework as the unit of comparison rather than varying the database access layer.

In practice, the most visible performance comparisons are *vendor benchmarks*: plentiful but narrow, published by the maintainers of the compared

products, confined almost exclusively to PostgreSQL, and settling on a single metric family rather than reporting throughput and the tail together. We found none that reports the p99 tail behavior that is widely held to matter for user-perceived latency under load [19], [22]. The peer-reviewed literature is sparser still, limited to at most three ORMs, confined to PostgreSQL, and, where it benchmarks PostgreSQL against MySQL directly, holding the access layer fixed rather than varying it [2], [15], [48], [62], [63] (Section 2 surveys each).

In the sources we searched (Section 2) this leaves a gap along four axes: (1) no access-layer comparison we found includes MySQL, and no PostgreSQL/MySQL benchmark varies the access layer; (2) no vendor-independent study spans the taxonomy broadly, since the broad-coverage comparisons are vendor-authored and academic work covers at most three libraries [2], [63]; (3) we found none that reports throughput and the p99 tail jointly, the sole vendor suite that pairs throughput with a percentile stopping at a single P95 at one load point [20]; and (4) none measures client-observed performance through an HTTP framework, academic studies instrumenting query execution inside the application process [63] and the Express-based work that measures the round trip comparing runtime optimizations on a single stack rather than access layers [28].

This paper addresses that gap with a single, vendor-independent benchmark: we serve an *identical* Express application through all nine access layers (two native drivers, Knex, and six ORMs: Drizzle, Prisma, Sequelize, TypeORM, Objection, MikroORM) over five CRUD-spanning access patterns rather than a measured production workload (point read, keyset range scan, deep/nested fetch, per-author aggregation, and insert; Section 3). Seven of the nine layers (all but the two engine-specific native drivers) run against *both* engines under an identical schema, dataset, and pool size; two tuned native-driver baselines (prepared-statement variants of `pg` and `mysql2`) mark what hand-tuned driver use achieves. Every (layer, engine, pattern) combination reports throughput (req/s) and saturated client-observed response-time p99 (at a 50-connection operating point) jointly, from 25 repeated runs, with a utilization-controlled open-loop tail measured below saturation. The study separates two estimands: the *default-configuration* effect of each layer used idiomatically (RQ1–RQ3), and a controlled *same-SQL* contrast that bounds the residual difference once every layer executes identical SQL, changing query formulation, protocol, preparation, round-trips, and mapping together rather than isolating any one. The full harness (adapters, seed data, an autocannon-driven [16] load runner, and an automated cross-check that all layers return *byte-identical* responses on the four non-mutating patterns) is released as an

open replication package.[1]

Three research questions structure the study, each scoped to the benchmarked implementations under the specified operating point:

RQ1 *(overhead):* How large is the throughput and saturated response-time-p99 overhead of the benchmarked implementations relative to the native-driver baseline, and how does it vary across the five access patterns?

RQ2 *(engine dependence):* Does the observed layer ordering transfer across PostgreSQL and MySQL, or is it engine-dependent?

RQ3 *(consequential patterns):* For which access patterns is the choice of implementation most consequential in this configuration?

This paper makes four contributions. The **primary** contributions are the harness and the benchmark design; the third is a configuration-specific snapshot of a fast-moving ecosystem, and the harness and design outlast the pinned versions:

- An **open, reproducible harness**: a uniform adapter contract, a deterministic seed, a matrix runner with repeated randomized-block runs and paired request streams, a physical write-state reset, a same-SQL control with server-side protocol evidence, a coordinated-omission-corrected open-loop generator with a utilization-controlled latency mode, and an automated **correctness cross-check** asserting **byte-identical** responses across layers on the four non-mutating patterns (Section 3).
- A **vendor-independent, broad-coverage benchmark design** spanning the taxonomy's three tiers (native driver → query builder → six ORMs), **dual-engine** (PostgreSQL and MySQL), measured at the **Express level**, reporting throughput and response-time p99 **jointly**, the combination missing from prior art.
- **Configuration-specific observations** on the pinned versions and this host: two not obvious in advance, that the query-engine ORM Prisma **ties, and on several patterns leads, the native driver** at a substantial application-CPU premium (Section 4), and that **whether writes are layer-bound depends on the engine** (visible only in a dual-engine

[1] Harness, deterministic seed, all adapters, raw per-cell measurements, and the table/figure-generating scripts: `https://github.com/mmiotk/express-db-access-performance`; release v1.4.1 is permanently archived at Zenodo, `https://doi.org/10.5281/zenodo.21373621`.

design); and a controlled **confirmation** that abstraction cost concentrates in relation-materializing patterns while the coarse read ordering is stable across engines.

- A practitioner-oriented **checklist of four pitfalls specific to access-layer benchmarking** (Section 5), surfaced during development, several caught by the correctness cross-check and the rest by performance analysis.

2 Related Work

We organize prior work along four axes on which our study differs from it (taxonomy coverage, engine coverage, whether throughput and tail latency are reported jointly, and vendor independence), summarized in Table 1. We located this work with a structured search of the ACM Digital Library, IEEE Xplore, Scopus, and Google Scholar for combinations of *ORM*, *query builder*, *database access layer*, *Node.js*, *Express*, *PostgreSQL*, and *MySQL* with *benchmark*, *performance*, *throughput*, and *latency* (2006–July 2026), followed by citation chaining, retaining empirical comparisons of relational access layers or engines and excluding non-empirical or non-relational studies.

2.1 Object-relational mapping and the access-layer spectrum

Access layers exist to bridge the object-relational impedance mismatch between the relational model and the object graphs an application manipulates [29], [46], spanning native drivers, query builders, and object-relational mappers following patterns such as Data Mapper and Active Record [21] (Section 1). Torres et al. [54] survey two decades of ORM patterns; the trade-off they document, developer productivity against a runtime cost that is hard to predict, is the one this paper quantifies. The same question recurs in the polyglot-persistence debate over non-relational stores [47]; we hold that choice fixed and vary only the access layer.

2.2 ORM performance and data-access anti-patterns

A parallel line of research studies ORM performance from the opposite direction: rather than benchmarking layers, it detects and repairs the inefficient SQL that mapping frameworks emit. The costliest pattern is the N+1 query problem (Section 1); it and related redundant-query anti-patterns are pervasive across real-world web applications [58], [59], and Shao et al. [50] give an

independent taxonomy and detector for database-access anti-patterns. Chen et al. quantify the performance impact of redundant data access in ORM-backed Java systems [10], extending earlier anti-pattern detection work [9]. A complementary strand shows the overhead is often *removable*: query synthesis lifts imperative accessor code into efficient SQL [12], lazy evaluation batches round trips [11], view-centric optimization rewrites request handlers [60], and second-level caching absorbs repeated reads [8]; the JavaScript analogue is mis-sequenced `async`/`await` serializing independent I/O [55]. Closest to our design, Lorenz et al. [41] show the best O/R mapping strategy depends on the database technology, van Zyl et al. [65] that ORM performance is highly sensitive to tuning, and a recent study adds an energy dimension [5]. This literature establishes *where* ORM overhead concentrates (relation-materializing, N+1-prone access) and that it is configurable rather than fixed, which our per-pattern results confirm; it does not compare drivers, query builders, and ORMs head-to-head under controlled load, our aim here.

2.3 Peer-reviewed access-layer comparisons

The peer-reviewed literature that measures access layers head-to-head is narrow. An early study by van Zyl et al. [64] weighed object databases against ORM tools. Colley et al. [15] give the earliest systematic cross-language account, benchmarking eight ORM frameworks across Java, C#, Python, and PHP against a common schema, but *deliberately excludes JavaScript*, leaving Node.js, and Express in particular, without an academic baseline for the better part of a decade. Three recent studies partially address this: one, in *Procedia Computer Science*, compares Prisma against optimized raw SQL over eight query patterns on a containerized PostgreSQL setup [62] (cited for its methodology; differing library versions, hardware, workload, and harness mean we neither build on nor dispute its absolute speedup figures); a second compares Prisma, TypeORM, and Sequelize across four relation types, also on PostgreSQL, by response time, memory, and CPU [2]; and the most recent instruments the same three ORMs on PostgreSQL inside a NestJS service, timing internal query stages rather than client-observed requests [63], independently observing the concurrency-dependent Prisma ranking flip we also report (worst at single queries, best under parallel load). A further Express-based study optimizes a single stack (pooling, batching, caching) rather than comparing access layers [28].

2.4 Database-engine, SQL/NoSQL, and Node.js performance

A second, largely disjoint line of work benchmarks the layers immediately below and above the access layer without varying it directly. Salunke and Ouda [48] compare PostgreSQL and MySQL head-to-head under standard OLTP workloads (the only dual-engine precedent found), but benchmark the *engines themselves*, bypassing any application-level access layer, so their work cannot say how a driver, query builder, or ORM would change it. In the broader engine-comparison line, work contrasting SQL and NoSQL databases [40] measures the store, not the access layer. On the Node.js side, Chaniotis et al. [7] established Node's viability as a web platform, Kuffel and Walter [34] tracked how its back-end performance drifts under sustained load, do Carmo et al. [6] compared back-end frameworks more broadly, and Sun et al. [52] characterize how the single-threaded event loop schedules the I/O every access layer relies on. None isolates the database access layer as an independent variable, let alone varies it across the taxonomy studied here.

2.5 Standard benchmarks and benchmarking methodology

Standard benchmarks define how database systems are measured: YCSB [18] for cloud-serving stores and the TPC-C/TPC-E suites for OLTP. These target the *engine* and its transaction mix, saying nothing about the driver, query builder, or ORM through which an application issues work (the variable this study isolates), but they set the methodological bar for controlled, repeatable load. A separate methodological literature explains why the vendor benchmarks' single-metric habit is a problem in its own right: Fruth et al. [22] and Raasveldt et al. [45] catalog pitfalls specific to database benchmarking (coordinated omission, warm-up, tool-induced distortion) that silently corrupt reported percentiles; Dean and Barroso [19] show why the tail, not the mean, determines user-perceived latency once a request fans out over many components, as an Express request serving a deep/nested fetch does; and Li et al. [39] trace the hardware, OS, and application sources of that tail. Load-testing surveys report such measurement is often ad hoc [30], [38], and repeatability studies find systems results frequently hard to reproduce [14], motivating a shared, reusable harness; the benchmarking literature stresses relevance, repeatability, and fairness [27] and the role of neutral benchmarks in advancing a field [51]. From this literature we take a design choice absent from every access-layer benchmark surveyed above: report throughput *and* tail latency (p99) for the same run, rather than a single metric family chosen

after the fact.

2.6 Practitioner and vendor benchmarks

A larger body of evidence comes from practitioners and vendors, though each source is authored by a party with a stake in one of the compared products. Drizzle publishes both a Northwind micro-suite spanning nine targets (`pg`, Knex, Kysely, MikroORM, TypeORM, Prisma, and Drizzle itself, pooled and unpooled) and an official k6-driven load test reporting requests per second, average latency, P95, and CPU [20]. Prisma's own site compares Prisma, Drizzle, and TypeORM by median query latency over 500 iterations, reporting neither throughput nor any latency percentile [43]. IMD-Bench, maintained by Gel Data (formerly EdgeDB), adds Sequelize and node-postgres and reports both throughput and latency on three CRUD-style operations [23]. Together these three sources cover more of the taxonomy than the entire peer-reviewed literature combined, yet share the same defects noted in Section 1: vendor authorship, PostgreSQL-only coverage, and a single reported metric family.

2.7 Positioning

In the sources searched through July 2026 (search protocol archived with the replication package), no study combined broad taxonomy coverage, both engines, joint throughput/tail reporting, and vendor independence; each covers at most two or three of these four properties (Table 1). This study combines all four (Section 1), closing the same engine gap separating access-layer studies from engine-only work such as [48].

3 Study Design

Our goal, in Goal–Question–Metric terms, is to *analyze* the relational database access layer of an Express.js service *for the purpose of* quantifying its performance cost *with respect to* throughput and response-time p99 *from the point of view of* an application developer choosing a layer *in the context of* PostgreSQL and MySQL back ends [4]. The design follows established guidelines for controlled experimentation in software engineering [32], [57], crossing seven portable access layers with both engines and all five patterns, plus two engine-specific native drivers and their tuned counterparts `pg-tuned` and `mysql2-tuned` (eleven layers: nine idiomatic, two tuned reference baselines); the full rationale is distributed with the

Table 1: Prior work on relational-database access-layer performance, positioned along the four axes on which this study differs from it. "Layers covered" aggregates native drivers, query builders, and ORMs into one count; "none" marks studies that vary the engine or framework but not the access layer.

Study	Layers covered	Engines	Metrics	Independent?
Colley et al. [15]	8 ORMs, 4 languages (no JS)	not stated	query latency	Yes
Procedia CS [62]†	Prisma vs. raw SQL	PostgreSQL	latency, CPU	Yes
JCCT [2]	3 ORMs	PostgreSQL	latency, memory, CPU	Yes
JCSI [63]	3 ORMs	PostgreSQL	per-stage latency	Yes
Salunke and Ouda [48]	none (engine only)	PostgreSQL + MySQL	throughput, latency	Yes
Drizzle [20]	9 (broad)	PostgreSQL	throughput, latency, P95	No (vendor)
Prisma [43]	3 ORMs	PostgreSQL	median latency only	No (vendor)
IMDBench [23]	4 ORMs + driver	PostgreSQL	throughput, latency	No (vendor)
This study	**9 idiomatic (+2 tuned)**	**PostgreSQL + MySQL**	**throughput + response-time p99**	**Yes**

†Cited for its methodology only; its absolute speedup figures are not comparable to ours given the differing versions, hardware, workload, and harness.

replication package (`METHODOLOGY.md`). Two estimands are kept separate throughout: the *default-configuration* estimand (RQ1–RQ3, Section 4), the cost a practitioner incurs from each layer used idiomatically, and a *same-SQL (raw-path)* estimand, what remains once every layer executes identical SQL through its raw facility, a compound contrast rather than a mechanism-isolating decomposition (Section 4).

3.1 Factors and treatments

We vary three factors. The *access layer* has eleven levels spanning the taxonomy: the native drivers `pg` and `mysql2`, their tuned counterparts `pg-tuned` (named prepared statements reused across calls) and `mysql2-tuned` (the binary protocol with server-side prepared statements via `execute()`), included as engine-specific tuned reference baselines rather than idiomatic treatments; the query builder Knex; and the ORMs Drizzle (a lightweight, query-builder-style ORM), Prisma, Sequelize, TypeORM, Objection, and MikroORM. We classify a library by its dominant interface, not its self-description (Section 1); Drizzle sits at the query-builder end of the ORM range and Objection layers an ORM over Knex, which the results bear out. The nine idiomatic layers cover the taxonomy's three tiers but are a representative cross-section, not an exhaustive catalogue; other libraries (for example Kysely, Slonik, and pg-promise) are omitted. Vendor independence here means authorship only: no evaluated library's maintainers were involved or invited to inspect the adapters, which does not remove the consequential choices any benchmark author makes (library selection, idiomatic-use criteria, schema and endpoints, pool size, allowed tuning); we disclose these through-

out and release the harness for inspection. The *engine* has two levels, PostgreSQL 18.4 and MySQL 9.7.1, and the *access pattern* has five levels, realized as HTTP endpoints (Table 2). The four native and tuned baselines are engine-specific (`pg`/`pg-tuned` run only on PostgreSQL, `mysql2`/`mysql2-tuned` only on MySQL); the other seven layers run on both engines, giving $4+7\times2=18$ layer $\times$ engine cells and $18\times5=90$ measured configurations. Supplement Table S27 summarizes which factors are held constant, which vary, and which are uncontrolled on this single virtualized host; the uncontrolled factors are why we report results for this configuration rather than as general library performance.

Table 2: The five access patterns and what each stresses.

Endpoint	Pattern	Stresses
`GET /posts/:id`	point read (PK)	per-query overhead
`GET /posts?before=`	keyset range scan	index seek + hydration
`GET /posts/:id/thread`	deep/nested fetch	join strategy, N+1 avoidance
`GET /authors/:id/summary`	aggregation	grouped counts / raw SQL
`POST /posts`	insert	write path

3.2 Objects: schema, workload, and data

The workload is a minimal blog domain (`authors` $1-*$ `posts` $1-*$ `comments` $*-1$ `authors`), exercising a one-to-many relationship and a two-hop join for the deep fetch. The five endpoints implement canonical CRUD access patterns [18] (Table 2): the keyset page issues `WHERE id < cursor ORDER BY id DESC LIMIT` n against the primary-key index rather than a large `OFFSET`; the deep/nested fetch returns a post with its author and every comment, each with its own comment-author; and the aggregation reports post count, comment count, and total views per author. The database is loaded from a deterministic seed (2,000 authors, 100,000 posts, 1,000,000 comments) by the native driver, independently of any layer under test, so every engine receives byte-identical data. The working set fits in memory, so measurements reflect access-layer cost rather than storage latency, per-request instruction and logging cost dominating over disk I/O [26]. Every request draws its target identifier uniformly at random from the full seeded range (posts 1–100,000, authors 1–2,000), so load spreads across the whole working set rather than repeatedly hitting one hot record, and the read endpoints measure warm access-layer cost. A skewed, production-like key distribution is a planned variant (Section 6).

3.3 The adapter contract and N+1 control

Every access layer implements the same factory interface (five query methods plus a teardown), so the Express server and the load runner are layer-agnostic. Each adapter returns an *identical result shape*, and each uses that layer's *recommended idiomatic* mechanism to avoid the N+1 query problem on the deep fetch: a hand-written join for the native drivers, the Knex query builder, and the query-builder-style Drizzle (the tuned baselines reuse the native drivers' join unchanged), and relation eager loading (`include`, `withGraphFetched`, `populate`, or `relations`) for the data-mapper ORMs. The comparison is therefore one of *idiomatic usage*, not a fixed query plan: a measured difference reflects the SQL each layer generates, its round-trip count, and how it materializes results. One asymmetry is inherent to any driver-versus-ORM comparison: the native side's idiom is hand-authored SQL, whereas the ORM side's idiom is the library's default. Server-side statement logging captures the exact SQL each layer emits, its round-trip count, and `EXPLAIN` plans for the native statements, released with the replication package; Supplement Table S2 tabulates the per-endpoint counts, which differ by design: the deep fetch takes one query in Sequelize and MikroORM, two in the native drivers, Knex, Drizzle, and TypeORM, and four in Prisma and Objection. The idiomatic API for each layer was fixed by a rule decided *before* measurement—the eager-loading API its official documentation presents first for the pinned version—so the treatment is a documented choice, not an editorial one; Supplement Table S16 and the package's `METHODOLOGY.md` record, per layer, that API with its **documentation page and version**, its round-trip count, the documented alternative we did *not* use, and that query logging, hooks, and validation are off. The six ORMs split both ways on the join-versus-select-in default; a loading-strategy sensitivity check (Section 4) switches join-default data-mapper ORMs to select-in wherever the alternative is a byte-identical drop-in. To separate a layer's execution machinery from its strategy choice, every adapter also implements a *same-SQL control*: identical two-statement deep-fetch SQL (and, per engine, identical `EXPLAIN` plans) with identical row mapping, executed through the layer's raw-SQL facility, mirroring the aggregation control (Section 4). Server-side statement capture, synchronized to a request marker so connection handshakes are excluded, confirms the two control statements are byte-identical across every layer on both engines; what differs is the wire protocol each driver negotiates for them: on PostgreSQL every layer uses the extended protocol with server-side prepared statements except MikroORM, which falls back to the simple protocol with client-side literal inlining, and on MySQL every layer uses the text protocol except `mysql2-tuned` and Prisma, which use the binary protocol with server-side

prepared statements. To ensure no adapter silently returns less, or issues N+1 queries, an automated cross-check (`bench/verify.mjs`) funnels every adapter's rows through shared canonical constructors and asserts full JSON byte equality against the native-driver baseline, on both engines, across 12 probes per adapter spanning the four non-mutating patterns and the same-SQL control, before any timing is collected. This check caught two defects during development (a fan-out aggregation bug and a driver result-shape mismatch; see Section 5).

3.4 Controls

Four controls isolate the access layer as the only variable. *(i) Pool size* is fixed at ten connections for every adapter, so the comparison measures access-layer overhead rather than pool tuning, a known web-application performance factor [25], [36]; for Prisma, whose default pool is otherwise derived from core count, this is pinned explicitly via the `connection_limit` URL parameter. Because the load generator opens 50 concurrent connections against this ten-connection pool, roughly forty in-flight requests are always waiting in each library's connection-acquisition queue; this queueing is deliberately part of what we measure and is held identical (same pool size, demand) across layers. A 200 ms connection-pool sampler corroborated this on the deep fetch: the native PostgreSQL cell ran with its ten-connection pool fully occupied (mean utilization $\approx$ 10 of 10) and a mean of about 32 requests pending in the acquisition queue (peaks of 39–40). *(ii) Identical query semantics*: each endpoint issues the same logical query across layers; aggregation uses a single correlated-subquery formulation everywhere (the documented raw-SQL path), so it measures layer overhead on an identical plan, not differences in generated SQL. *(iii) Write isolation*: because the insert endpoint mutates the dataset, every cell resets it before warm-up and before every measured run, not merely once per cell; the write endpoint is additionally measured in its own dedicated server boot, taken after the database's physical state is rebuilt rather than merely emptied by row deletion (PostgreSQL: drop and recreate the database from a seeded template; MySQL: delete the benchmark rows, rebuild the table with `OPTIMIZE TABLE`, and re-pin `AUTO_INCREMENT`), so sequences, physical layout, and purge state cannot drift across replicates or leak between adapters. *(iv) Observer separation*: the HTTP load generator runs in a process separate from the server.

3.5 Measurement procedure

Load was generated with `autocannon` [16], an HTTP/1.1 load tester issuing requests continuously at fixed concurrency and recording a latency histogram; it is a closed-loop (constant-concurrency) generator [49]. We measured each of the 90 configurations as 25 *repeated runs*. For each replicate we booted a fresh server process, opened 50 concurrent connections, ran a fifteen-second warm-up pass whose measurements were discarded, and then took one twelve-second measured run. The warm-up length follows a separate cold-boot measurement for a fast, middling, and the slowest PostgreSQL layer: every layer reached and sustained at least 95% of steady-state throughput within 12 s (Prisma by 5 s, native by 9 s, MikroORM by 12 s), absorbing JIT compilation, pool fill, and plan caching, since managed runtimes do not always reach a stable steady state promptly [3]. The experimental unit is the freshly booted server run for one (layer, engine, endpoint) cell in one replicate; individual HTTP requests within a run are subsamples, not independent units. Booting a fresh process per replicate prevents persistent application-process state (JIT state, pool contents, heap) from carrying across replicates, though not full statistical independence: all runs share one host and one short campaign, so replicate index and measurement order are treated as blocking factors, not sources of independence. Cell order was reshuffled independently within each replicate; a forward-versus-reversed-order check on the write endpoint (5 replicates per direction) found no detectable history effect (reversed-to-forward throughput ratio across all 18 cells: 0.970–1.021, median 1.004). Within a replicate, every layer and engine is driven by the identical request-id sequence, generated by a PRNG seeded from the (endpoint, replicate) pair; this paired-stream design removes between-layer sampling noise as a confound, and the first ids of every stream are archived in the replication package. We report the median of the 25 replicate values; 449 of the 450 dedicated write-endpoint boots in the primary campaign completed normally, the exception being knex/PostgreSQL, whose replicate-16 boot hit a health-check timeout immediately after the write-state reset, leaving that cell with 24 of 25 repeated runs (the runner records and skips such failures rather than retrying silently). Each replicate yields throughput (requests per second) and latency percentiles p50, p90, p97.5, and p99. During measured runs we sampled server CPU and peak resident memory from `/proc` over the full process tree (parent plus descendants, so an in-process engine or spawned helper cannot escape measurement), with database and load-generator CPU sampled separately; a `perf_hooks` GC observer (event count, total pause time) and the 200 ms connection-pool sampler ran inside

the server, polled through a `/stats` endpoint after each run. Inserts were measured under each engine's *default* durability configuration (PostgreSQL `fsync=on`, `synchronous_commit=on`, `full_page_writes=on`; MySQL `innodb_flush_log_at_trx_commit=1`, `sync_binlog=1`, `log_bin=ON`), the regime a deployment runs unless an operator deliberately relaxes it. Each insert is an autocommit single-statement `INSERT` under each engine's default isolation level (PostgreSQL Read Committed, MySQL Repeatable Read); the transactional variant wraps a post and its five comments in one transaction. A labelled secondary run instead relaxed every one of those settings on both engines (10 replicates, write endpoint only) to isolate the durability mechanism, compared with the default regime in Section 5 (Supplement Table S3). A fixed-payload `/baseline` endpoint returning a fixed, seed-realistic thread-shaped payload measured the shared Express-plus-serialization floor (bounding framework and serialization cost, not per-request object construction): 10,909 (PostgreSQL server) and 10,921 (MySQL server) req/s, roughly 3× the fastest deep-fetch cell, so the framework floor leaves between-layer spreads visible rather than compressed. To characterize behavior under load, we swept the deep/nested fetch (the most layer-sensitive pattern) across connection counts from 1 to 256 for every layer and engine, since throughput and contention scale non-linearly with concurrency [61]; a separate equal-CPU-budget control (1, 2, or 4 cores) attributes part of that scaling directly to CPU (Section 4). A native, `undici`-based constant-arrival generator drove the deep fetch under a fixed offered rate (250–4,000 req/s) rather than fixed concurrency, measuring latency from each request's intended send time and clipping timed-out requests into the distribution at their cutoff; this coordinated-omission-corrected design [33], [53] independently validates the closed-loop tail results (Section 4).

3.6 Analysis

We separate a small set of *primary* outcomes, which carry the paired inference and the research-question claims, from *secondary* and *exploratory* outcomes, which are reported descriptively as controls and sensitivity analyses (Table 3). The primary outcomes are throughput and the closed-loop p99 tail on the five patterns against both engines at the fixed operating point; everything else bounds or contextualizes them.

Because absolute throughput is hardware-specific, we analyze *relative* differences across layers within an engine. We quantify a pattern's sensitivity to the access layer by its *spread*: the idiomatic native driver's (`pg` or `mysql2`; the tuned baselines are a separate tuned reference, not the spread's

Table 3: Outcomes and their analysis role. The primary outcomes carry the paired inference and the research-question claims; the secondary and exploratory outcomes are reported descriptively as controls and sensitivity analyses that bound or contextualize the primary results, and are not treated as additional confirmatory hypotheses.

Role	Outcome	Analysis
Primary	Throughput (req/s), 5 patterns × 2 engines, idiomatic layers, 50-connection operating point	Paired permutation, Wilcoxon, bootstrap median CI; spread & rank
Primary	Closed-loop p99 tail latency, same cells	Paired permutation, Wilcoxon, per-cell bootstrap CI on p99
Secondary	Same-SQL (raw-path) contrast	Bounds the residual (compound) same-SQL difference
Secondary	Sub-saturation open-loop tail (coordinated-omission corrected)	Near-intrinsic latency companion to the saturated primary p99
Secondary	Layer×engine interaction (per pattern)	Blocked permutation (F on D)
Secondary	Combined app+db CPU efficiency, equal-CPU control	End-to-end compute view; operational confirmation of the CPU trade
Explor.	Aggregation fan-out; OFFSET vs. keyset; pool-size sweep; durability & isolation sensitivity; RSS	Descriptive; pitfall illustration and robustness checks

anchor) median throughput divided by that of the slowest layer on that pattern (native-relative, so comparable across patterns even where an ORM exceeds the native driver). We report medians over the 25 repeated runs, the coefficient of variation (CV) as a stability signal, following recommended practice [24] and using enough replicates to bound variance [35] (Section 4), and a bootstrap 95% CI for the median. The design is a randomized block: within each replicate every layer runs the identical request stream, so two layers' throughput samples are matched by replicate and must be compared with *paired* tests. To test whether adjacently ranked layers are distinguishable rather than an artifact of measurement noise, we compare each adjacent pair on the deep/nested fetch on their per-replicate throughput ratios with a two-sided paired sign-flip permutation test (20,000 permutations, so the smallest attainable p is $1/(20{,}001)$), cross-checked with the Wilcoxon signed-rank test. We report the geometric-mean paired ratio with a paired bootstrap 95% CI and the paired dominance (fraction of replicates in which the faster layer wins). In every test the unit is the *run-level* statistic (one throughput or p99 value per repeated run, 25 per cell), never the individual request; all confidence intervals are *percentile* bootstraps resampling replicate indices (preserving the pairing), from a fixed seed (`mulberry32(0x57a75)`) so every resample is reproducible. Because p99 is reported at millisecond resolution and contains ties, the permutation test keeps zero log-ratios and the Wilcoxon test drops them with the standard tie correction. Because the seven adjacent pairs are defined by the *observed* ranking rather than preregistered, we treat

their significances descriptively (post-hoc adjacent-rank comparisons), applying a Bonferroni correction as a conservatism check rather than claiming a confirmatory family. For the pg–Prisma pair we additionally report a paired two-one-sided-tests (TOST) equivalence result against a ±5% margin on the log-ratio (justified below). The layer×engine interaction is tested per pattern with a blocked permutation test: for each layer and replicate we form the paired PostgreSQL-minus-MySQL log-throughput difference $D_{\ell,i}$ and, under the null of no interaction, permute the layer labels of $\{D_{\ell,i}\}$ *within* each replicate i (20,000 permutations); the test statistic is the between-layer F of a randomized-block ANOVA on D with replicate as the block. Rank agreement across engines (Spearman, Kendall) is reported descriptively over the seven evaluated layers — systems under study, not a sample from a population — so we do not attach population confidence intervals to those correlations. Reporting nonparametric tests with standardized effect sizes follows recommended practice for randomized performance experiments in software engineering [1], [13]. The ±5% equivalence margin is a deployment-relevance threshold: a throughput difference under 5% would not change a capacity-planning decision (for example, how many application instances to provision for a target load), and it is comfortably within the run-to-run and day-to-day variation typical of a virtualized host (for reference, this study's own run-to-run CV reaches 7.6%). We chose it before the analysis but did not preregister it, and the equivalence conclusion is not sensitive to the exact value within a few percent.

3.7 Experimental setup

All measurements were taken on 2026-07-12 and 2026-07-13 on a single host: the primary matrix ran overnight across both dates, and the order-invariance, durability, same-SQL, open-loop, fan-out, equal-CPU, and concurrency-sweep runs followed on 2026-07-13 (a virtualized Intel Xeon Gold 6140 instance, 16 vCPUs, single NUMA node, 126 GB RAM; Linux 6.17; Node.js 24.16.0), with the database engines running locally. The pinned versions (Supplement Table S11) were selected and installed on the measurement date, recorded from the lockfile and an automated environment capture (replication package), and frozen for the study, since access-layer performance is version-sensitive; later point releases (for example Node.js 24.17.0) are deliberately excluded. We pin Express 4 rather than Express 5; since it is shared by every cell, it shifts the common request-handling floor rather than the between-layer comparison to first order, though a different version could interact with a layer's scheduling, and re-running the harness on Express 5 is a one-line change.

3.8 Replication package

The harness — the Express application, the adapter contract, all nine idiomatic adapters and the two tuned baselines, the deterministic seed, the `autocannon` matrix runner, the correctness cross-check, and the scripts regenerating every table in Section 4 — is released so the full matrix can be re-run with one command against a containerized or local PostgreSQL and MySQL.

4 Results

The tables report throughput (requests per second, higher is better) and tail latency (p99, milliseconds, lower is better) per access layer and engine. The two consequential patterns are in the main text (the deep/nested fetch, Table 4, and the insert, Table 5); the point read, keyset range scan, and aggregation are in Supplement Tables S12, S13, and S15. All numbers come from the run of Section 3. Each native driver is reported both idiomatically (`pg`, `mysql2`) and tuned (`pg-tuned`, `mysql2-tuned`), the tuned rows marking the throughput ceiling hand-tuned driver use reaches, not a level every idiomatic layer attains without code changes. The comparison of interest is *relative* across layers within an engine (Section 3); two estimands recur below: the *default-configuration (idiomatic) effect* (RQ1–RQ3) and the *same-SQL effect* (Supplement Table S14).

Table 4: Deep/nested fetch: throughput (req/s, higher is better; median with 95% bootstrap confidence interval over the 25 repeated runs) and response-time p99 (ms, lower is better) by access layer and engine.

		PostgreSQL		MySQL	
Layer	Category	req/s [95% CI]	p99	req/s [95% CI]	p99
pg	native-driver	3741 [3687–3782]	20	–	–
mysql2	native-driver	–	–	2405 [2371–2423]	29
pg-tuned	native-tuned	3813 [3761–3851]	18	–	–
mysql2-tuned	native-tuned	–	–	2436 [2407–2472]	25
knex	query-builder	2491 [2442–2534]	27	2050 [2031–2065]	30
drizzle	orm-lightweight	2080 [2048–2101]	31	1446 [1437–1461]	43
prisma	orm	3685 [3653–3712]	18	3701 [3680–3717]	18
sequelize	orm	989 [978–1007]	60	893 [883–902]	65
typeorm	orm	1327 [1310–1343]	45	1170 [1160–1195]	50
objection	orm	1037 [1028–1059]	55	860 [848–864]	67
mikroorm	orm	571 [562–576]	106	535 [524–541]	113

Table 5: Insert: throughput (req/s, higher is better; median with 95% bootstrap confidence interval over the 25 repeated runs) and response-time p99 (ms, lower is better) by access layer and engine.

		PostgreSQL		MySQL	
Layer	Category	req/s [95% CI]	p99	req/s [95% CI]	p99
pg	native-driver	6064 [5957–6235]	11	–	–
mysql2	native-driver	–	–	809 [796–816]	138
pg-tuned	native-tuned	6280 [6105–6399]	12	–	–
mysql2-tuned	native-tuned	–	–	808 [794–814]	135
knex	query-builder	4693 [4652–4819]	14	795 [783–802]	135
drizzle	orm-lightweight	4230 [4195–4253]	14	796 [793–805]	138
prisma	orm	5465 [5437–5546]	14	714 [709–726]	154
sequelize	orm	2668 [2647–2692]	21	772 [767–786]	135
typeorm	orm	2595 [2566–2673]	24	732 [718–748]	152
objection	orm	3926 [3767–3946]	15	787 [779–794]	140
mikroorm	orm	1571 [1560–1594]	37	663 [655–672]	153

4.1 RQ1: overhead relative to the native baseline

The native `pg` driver, or its tuned counterpart, leads only on the PostgreSQL point read and insert. On every other pattern-engine combination, Prisma leads outright, including over the tuned native baseline: the range scan and aggregation on both engines, and the point read and deep fetch on MySQL (Table 4, Supplement Table S15 for aggregation, Supplement Tables S12–S13 for the reads). It does so at roughly five times the application-side CPU of every other layer (498% on the deep fetch; Supplement Table S10), a CPU-for-throughput trade rather than a free win.

`pg-tuned` edges idiomatic `pg` on the point read, insert, and deep fetch by a few percent (reusable prepared statements); `mysql2-tuned` is within noise of idiomatic `mysql2` on every pattern (at most 0.8%) except the deep fetch, where the binary protocol and server-side prepared statements pay off only once a request issues more than one round trip.

On the **point read**, the native-relative spread is 3.06× on PostgreSQL and 2.36× on MySQL, where Prisma, not the native driver, is fastest (38% above `mysql2`; Supplement Table S12).

On the **deep/nested fetch** (the pattern that assembles a post, its author, and all comments with their authors), the native-relative spread is the widest measured: 6.55× on PostgreSQL (`pg` 3,741 down to MikroORM's 571 req/s; CI [6.46–6.65]) and 4.50× on MySQL (`mysql2` 2,405 down to MikroORM's

535; CI [4.41–4.59]). Prisma sits close behind `pg` on PostgreSQL (3,685) and ahead of `mysql2` on MySQL (3,701, 54% above native). The comparison is paired (Section 3): `pg` is faster in 16 of 25 replicates, by a geometric-mean ratio of 1.02 (paired 95% CI [1.00–1.04]; Table 7), a difference the paired sign-flip permutation test does not resolve as significant ($p = 0.058$; the Wilcoxon signed-rank test agrees, $p = 0.08$). A paired two-one-sided-tests (TOST) equivalence test at an a-priori ±5% margin shows the two meet the ±5% equivalence criterion (ratio 1.02, 90% CI [1.004–1.037], inside the ±5% band), and Prisma's p99 is the lower of the two (18 ms vs. 20 ms); we therefore read the top of the PostgreSQL deep-fetch ranking as a practical tie. Knex (2,491 / 2,050) and the lightweight ORM Drizzle (2,080 / 1,446) remain competitive on both engines, whereas the data-mapper ORMs (TypeORM, Sequelize, Objection, and especially MikroORM) trail furthest.

Aggregation overhead shrinks once every layer is forced onto the same raw correlated-subquery plan, but Prisma still leads outright on both engines, including over the tuned native baseline (76% above `mysql2` on MySQL; Supplement Table S15). Excluding Prisma, the native-relative spread is only 1.86× on PostgreSQL and 1.42× on MySQL: abstraction is cheap when a layer merely forwards a query, and costly only when it must materialize a nested object graph.

4.2 Same-SQL control

The idiomatic deep fetch lets each layer choose its own query strategy, mixing raw SQL execution with everything else a layer does. The *same-SQL control* (Section 3) bounds this residual difference: server-side capture confirms the statement text is byte-identical across layers on both engines, while the wire protocols differ per layer; this control is therefore *same SQL, protocol disclosed*, not *same plan*. The contrast is composite: switching from a layer's idiomatic relational-fetch path to the common raw-SQL-and-shared-mapping path changes its query strategy, query count, prepared-statement behavior, the raw versus ORM API, and result marshalling together, so the residual difference is bounded but no single factor is isolated.

On the identical SQL, the deep-fetch native-relative spread collapses from its idiomatic 6.55× (PostgreSQL) and 4.50× (MySQL) to 1.47× and 1.27× (Supplement Table S14). Prisma's raw path leads on both engines, consistent with its pipelined query engine; the new low end is Sequelize's raw facility (`sequelize.query`), which we attribute to dispatch and marshalling overhead in the raw API itself, not SQL execution. MikroORM shows the largest strategy gap: its idiomatic path reaches only 0.20 (PostgreSQL) and 0.25 (MySQL) of its own same-SQL throughput, a 5.0× and 4.1× idiomatic-to-

raw gap. The fixed-payload Express-plus-JSON floor (Section 3) sits well above every same-SQL cell, so the framework floor does not account for the collapse. A loading-strategy sensitivity check (Supplement Table S18) tests whether these deficits are merely a default-strategy artifact: switching the join-default data-mapper ORMs (Sequelize, MikroORM) to select-in, where the alternative is a byte-identical drop-in, makes them slower (0.68–0.75×), so their deficit is intrinsic; conversely, Objection's batched-select-in default is 1.6× slower than a single join on MySQL, so part of *its* deficit there is a strategy choice rather than an intrinsic cost.

4.3 RQ2: engine dependence of the ranking

The relative ordering of access layers is largely, but not uniformly, stable across engines: the Spearman rank correlation between the PostgreSQL and MySQL orderings of the seven portable layers (descriptive; Section 3) is $\rho = 0.89$ on the point read, 0.89 on the range scan, 0.96 on the deep fetch, and 0.89 on aggregation (Kendall's τ 0.81–0.90 on all four). The **insert** ranking is the exception ($\rho = 0.43$, $\tau = 0.43$; Supplement Table S26 gives the per-layer ranks that back these correlations). Prisma illustrates why: fastest ORM on PostgreSQL inserts (5,465 req/s, ahead of Objection's 3,926) but second-slowest on MySQL (714, above only MikroORM's 663) — a rank flip invisible to a single-engine benchmark. The *magnitude* of a layer's cost also depends on the engine: a blocked permutation test on the paired PostgreSQL-minus-MySQL log-throughput difference (Section 3) finds the layer×engine interaction significant on all five patterns (observed statistic above every one of the 20,000 permutations). Because the cross-engine differences are large and systematic, that p reaches its floor (1/20,001) and conveys little about magnitude, so we read the interaction through its effect size: the per-layer cross-engine (PostgreSQL/MySQL) throughput ratio spans a narrow 1.0–1.6 on the reads but a wide 2.4–7.7 on the insert, so single-engine results are not portable in degree, most sharply for writes, even where portable in order.

Prisma also leads the **keyset range scan** on both engines, ahead of the tuned and idiomatic native drivers, with a native-relative spread of 4.89× on PostgreSQL and 4.43× on MySQL (Supplement Table S13), confirming that the earlier collapse observed under `OFFSET` pagination was a workload artifact rather than an engine or layer property (Section 5); RQ3 below attributes most of that spread to a single layer.

Inserts are the pattern where the engine dominates most visibly, and where the durability regime matters to that claim: all headline write numbers here are under each engine's *default* durability configuration (Section 3). On MySQL, insert throughput is low and tightly clustered across all nine

layers (663–809 req/s, a 1.22× spread, CI [1.20–1.24]; Table 5) and its p99 sits at roughly 135–155 ms for every layer. Direct `performance_schema` instrumentation attributes this floor to the commit path: the redo-log and binlog flush takes 0.57–0.69 ms per insert, 38–40% of each insert wall time and nearly identical across the native driver, query builder, and slowest ORM. The flush partly overlaps other work, so relaxing durability lifts throughput by 19–24% rather than by its full wall-time share, but the floor is shared and engine-side, not layer-specific (Supplement Table S19). PostgreSQL, by contrast, spreads 3.86× (`pg` 6,064 down to MikroORM 1,571, CI [3.77–3.98]). A default-versus-relaxed comparison (Supplement Table S3) shows this is not a durability artifact: at 50 concurrent connections group commit amortizes most of the per-commit flush, so relaxing durability buys at most 14% on PostgreSQL and 19–24% on MySQL, and the engine contrast is, if anything, slightly larger under default durability. On MySQL, insert throughput is therefore governed mainly by the engine rather than the access layer; on PostgreSQL the access layer remains the larger factor.

4.4 RQ3: which patterns matter most

Ranking the five patterns by native-relative spread on PostgreSQL gives deep fetch (6.55×) > range scan (4.89×) > insert (3.86×) > point read (3.06×) > aggregation (1.86×). The same deep > range > point > aggregation order holds on MySQL (4.50×, 4.43×, 2.36×, 1.42×); only the insert moves, from third place on PostgreSQL to last on MySQL (1.22×), the pattern where MySQL's engine floor, not the access layer, sets the ceiling (RQ2 above). The deep/nested fetch is thus the most consequential pattern on both engines and aggregation the least, once every layer runs the same raw-SQL plan (the range scan's large nominal spread is driven disproportionately by MikroORM alone).

The primary deep-fetch workload uses posts with roughly ten comments on average. A fan-out sweep across 0, 1, 10, 50, 100, and 500 comments (both engines, medians of eight repeated runs) shows the native-relative spread is *roughly constant* with graph size—about 5.5–6× on PostgreSQL and 3.5–4× on MySQL across the whole range—so on this schema the deep-fetch deficit behaves as a fixed multiplier rather than one that grows with the graph. (An earlier three-run sweep had suggested a growing spread; that did not survive replication, so we report the fan-out as exploratory.)

4.5 Tail latency

We report the closed-loop p99 for every cell (p50, p90, p97.5 in the replication package), measured at the 50-connection operating point against the fixed ten-connection pool (Section 3), so the tail includes connection-acquisition queueing — the tail a deployment sees when driven to saturation, not an estimate of intrinsic per-request latency; the open-loop experiment below reports the sub-saturation tail. Under saturation, tail latency tracks throughput: on the deep/nested fetch, PostgreSQL's p99 ladder runs `pg` 20, Prisma 18, Knex 27, Drizzle 31, TypeORM 45, Objection 55, Sequelize 60, and MikroORM 106 ms (Table 4), monotonic below the `pg`/Prisma tie — a penalty visible in the saturated tail that throughput-only vendor benchmarks miss. The tail differences carry the same paired inference as throughput: on the deep fetch six of the seven adjacent p99 pairs separate under the paired permutation test with narrow per-replicate bootstrap 95% CIs (the ladder above, replication package), and across the read patterns five to six of seven separate on each engine. On MySQL inserts, by contrast, no adjacent p99 pair separates (every layer at 135–155 ms), the tail counterpart of the throughput engine floor.

These closed-loop values come from a constant-concurrency generator; to check the tail ranking is not an artifact of that model, we drove the deep/nested fetch on PostgreSQL under the coordinated-omission-corrected open-loop harness (Section 3; timed-out requests, a 10 s hard cap, clipped into the distribution; full sweep in Supplement Table S6). Below saturation the corrected tails are flat and small: `pg` and Prisma hold p99 at 2–4 ms up to 2,000 req/s, both sustaining the full offered rate, and Knex's p99 is still only 33 ms at 2,000. Beyond each layer's capacity the tail collapses rather than degrading gracefully: at the 4,000 req/s tier `pg` and Prisma still clear most requests (corrected p99 1.2–2.1 s) while Knex reaches 9.4 s, and the data-mapper ORMs collapse far earlier—Sequelize at 2,000 req/s and MikroORM at 1,000, each shedding a large share of requests to timeout (Supplement Table S6). We call a layer's behavior at an offered rate a *collapse* when it achieves under half the offered rate with a large share of requests timing out. By that criterion the ordering matches the closed-loop result: the data-mapper ORMs enter unstable queueing as soon as offered load crosses their capacity, while the lightweight layers degrade gradually. The same open-loop sweep on MySQL (Supplement Table S17) reproduces the pattern—flat sub-saturation tails and capacity-crossing collapse—so the saturated-tail ordering is engine-robust. That saturated tail tracks capacity, not intrinsic latency. Offering each layer 50/70/85/95% of *its own* saturating throughput (coordinated-omission-corrected, five runs, both engines; Supple-

ment Tables S21–S22), the tails converge: at 50% utilization every layer holds p99 near 2–4 ms on PostgreSQL, regardless of where it sits in the saturated ladder, and rises only as it nears its own capacity. At matched utilization the large saturated gap (`pg` 20 ms versus MikroORM 106 ms) therefore largely dissolves. The single-rate open-loop cells above are single runs, so near-knee point values are indicative; the utilization sweep and the flat-then-collapse ordering are what reproduce.

4.6 Measurement stability and significance

Across the 40 PostgreSQL cells the coefficient of variation of throughput over the 25 repeated runs is at most 7.6% (`pg`, insert; Supplement Table S9), and across the 40 MySQL cells at most 6.6% (Knex, insert; Supplement Table S1) — well below the between-layer spreads reported above. Every measured request in the primary matrix succeeded: HTTP errors, connection-acquisition timeouts, and non-2xx responses were zero across all 90 cells' measurement windows (nonzero timeouts arise only in the deliberate open-loop overload runs below). One cell, Knex/PostgreSQL/insert, has $n = 24$ (the one write boot that failed its post-rebuild health check; Section 3); the tabulated adjacent-pair tests are on the deep fetch, where every cell has the full 25 replicates, so this write cell does not enter them. As an environmental-robustness check we restarted both database engines (cold buffer pools, fresh connections) and re-measured a representative deep-fetch subset (`pg`/`mysql2`, Prisma, MikroORM): the layer ranking reproduced the primary campaign on both engines, with absolute throughput 3–23% lower eight days later (larger on PostgreSQL), day-to-day host drift the relative analysis absorbs (Supplement Table S20).

Comparing each adjacently ranked pair on the deep/nested fetch with the paired permutation test (method in Section 3; Table 7), six of the seven pairs separate with the observed statistic above every one of the 20,000 permutations ($p = 1/20{,}001 \approx 5 \times 10^{-5}$) and the faster layer ahead in nearly every replicate (dominance 1.00, except objection > sequelize at 0.88), all with ratios at least 1.05; the Wilcoxon test agrees throughout (replication package). Because the pairs are adjacent ranks of the *observed* ordering rather than preregistered, we treat these significances descriptively; a Bonferroni correction leaves all six far below any conventional threshold. The seventh, `pg` and Prisma, is the exception treated above (practical tie, $p = 0.058$, within $\pm 5\%$), so every step below it is both statistically and practically supported.

4.7 Scaling with concurrency

A concurrency sweep from 1 to 256 connections (Supplement Figure S1) confirms the coarse three-band ordering holds at every operating point above about 32 connections, and that `pg`/`pg-tuned` lead Prisma at every concurrency from 8 upward—so Prisma's near-native parity at the 50-connection operating point does not extend to an outright lead at higher concurrency; that operating point sits above the knee for every layer, so the RQ1 finding is not an artifact of a single point.

4.8 Resource footprint

Throughput is not the only cost (Table 6). Prisma's near-native or leading throughput on several patterns comes at a much higher CPU cost: measured at the *same* ten-connection pool as every other layer, its Node.js process draws 498% (about five cores) on the deep fetch against the ≈ 100% (one core) of the other layers, because Prisma runs a multi-threaded query engine in-process (Section 3), a property of the layer rather than of a larger connection pool; the multiplier is pattern-dependent (Prisma's aggregation cell draws only ≈ 200%).

Normalizing throughput by CPU cost must be read at the right tier. Counting only *application*-process CPU on the deep fetch, requests per CPU-second are `pg` 3,741 down to Prisma 740, so Prisma ranks second-to-last. That figure understates the compute of layers that push work to the database: `pg`'s hand-written join drives about 3.6 database cores while Prisma's engine keeps more work in the application tier. Counting *combined* application-plus-database CPU per completed request (Table 6), `pg`'s efficiency advantage over Prisma shrinks from about 5× to about 1.4× (813 vs. 596 req per combined-CPU-second); Prisma remains among the least efficient layers end-to-end, but the gap is a small multiple. An equal-CPU control corroborates the application-tier trade: confining the server process with `taskset` to 1, 2, or 4 cores (database and load generator pinned to disjoint cores; Supplement Table S4), `pg` reaches its full deep-fetch throughput on a single core (3,663, 3,277, and 3,700 req/s at 1, 2, and 4 cores), whereas Prisma needs roughly four cores to approach that level (896, 1,726, 3,025), 4.1× slower than `pg` on an equal one-core application budget; MikroORM stays low regardless of core count (463, 577, 542). The same trade governs a multi-worker deployment. Scaling each layer to one, two, and four Express workers (a shared-port `node` cluster; Supplement Table S23), the single-pool layers scale near-linearly on the otherwise-idle cores while Prisma—whose engine already uses several cores at one worker—plateaus: `pg` and Prisma are at parity at one worker

Table 6: Resource footprint on the deep/nested fetch (PostgreSQL, medians over 25 replicates): application-process-tree CPU and database-server CPU (% of one logical core), end-to-end throughput per combined (application+database) CPU-second, and peak resident memory. Prisma draws about five times the application CPU of the other layers at the fixed ten-connection pool; counting the database side as well narrows its end-to-end efficiency gap to the native driver to about 1.4×. An equal-CPU control (Supplement Table S4) confirms the trade operationally: on a one-core application budget `pg` reaches 3,663 req/s to Prisma's 896. Per-layer efficiency and equal-CPU detail are in Supplement Tables S5 and S4.

Layer	app CPU%	db CPU%	req/comb-CPU-s	RSS (MB)
pg	100	360	813	205
knex	100	245	722	204
drizzle	100	205	682	250
prisma	498	120	596	266
sequelize	100	50	659	248
typeorm	105	120	590	271
objection	100	50	691	211
mikroorm	100	45	394	255

(3,308 vs. 3,141 req/s on PostgreSQL) but `pg` overtakes Prisma by 1.6× at four workers (10,178 vs. 6,439), and `mysql2` overtakes Prisma by 1.4× on MySQL. Prisma's throughput parity is thus specific to a deployment that leaves cores idle; where every layer is scaled out to use them, the native driver leads.

Memory differences are smaller (Table 6): the native driver, Knex, and Objection are lightest (204–211 MB peak RSS), while the remaining layers use more (248–271 MB). Per-instance resource use governs how many replicas a given load requires when sizing horizontally scaled deployments [56].

Table 7: Paired comparison of adjacently ranked access layers on the deep/nested fetch (PostgreSQL), over the 25 repeated runs. Because every layer runs the identical per-replicate request stream, layers are compared on their per-replicate throughput ratios: median throughput (req/s) with bootstrap 95% CI, the geometric-mean paired ratio A/B with a paired bootstrap 95% CI, paired dominance (fraction of replicates in which A exceeds B), and the two-sided paired sign-flip permutation p (20000 permutations; a value of 5.0×10^{-5} is the resolution floor $1/(B+1) = 1/20{,}001$, i.e. the observed statistic exceeded every permutation; the Wilcoxon signed-rank test agrees, replication package).

Comparison (A > B)	med. A [95% CI]	med. B [95% CI]	ratio A/B [95% CI]	dom.	perm. p
pg > prisma	3741 [3687–3782]	3685 [3675–3712]	1.02 [1.00–1.04]	0.64	0.0584
prisma > knex	3685 [3653–3712]	2491 [2442–2534]	1.47 [1.45–1.50]	1.00	5.0×10^{-5}
knex > drizzle	2491 [2442–2534]	2080 [2048–2103]	1.20 [1.18–1.22]	1.00	5.0×10^{-5}
drizzle > typeorm	2080 [2048–2103]	1327 [1310–1343]	1.55 [1.53–1.57]	1.00	5.0×10^{-5}
typeorm > objection	1327 [1315–1343]	1037 [1028–1059]	1.28 [1.27–1.30]	1.00	5.0×10^{-5}
objection > sequelize	1037 [1026–1059]	989 [978–1007]	1.05 [1.03–1.06]	0.88	5.0×10^{-5}
sequelize > mikroorm	989 [978–1007]	571 [562–576]	1.74 [1.71–1.77]	1.00	5.0×10^{-5}

5 Discussion

Implications for practice. RQ1 and RQ3 agree that abstraction cost is specific to the *access pattern*, concentrating where the layer must hydrate rows into objects and resolve associations rather than merely issue SQL (the object-relational impedance mismatch [29] made concrete), where the data-mapper ORMs trail the native driver by the widest spreads (Section 4). The same-SQL control bounds this cost rather than localizing it to any one mechanism (Section 4). Round-trip count alone does not set throughput: Prisma and Objection both issue four deep-fetch queries (Supplement Table S2), yet sit at opposite ends of the default-configuration ranking, while Knex and Drizzle sit in a consistent middle ground on every pattern.

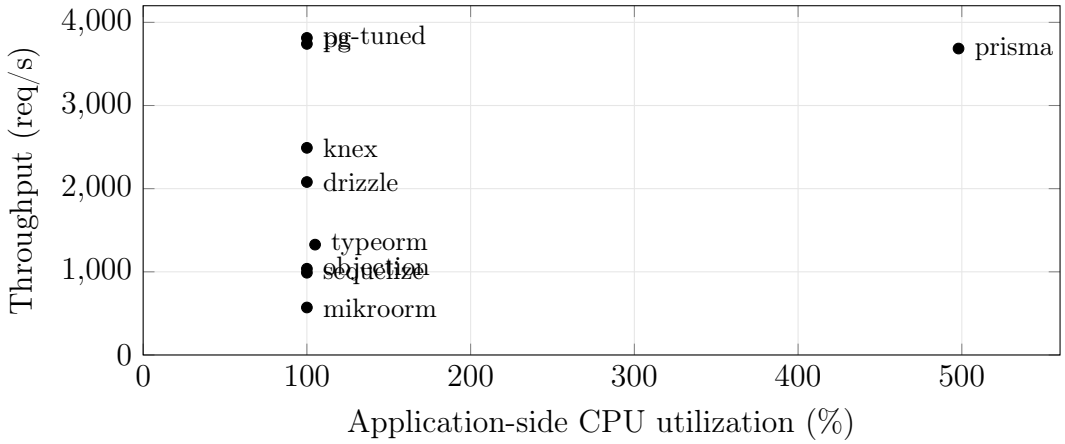

Figure 1: Throughput versus application-side CPU on the deep/nested fetch (PostgreSQL, ten-connection pool, medians of 25 replicates). Prisma reaches native-tied throughput at roughly five times the CPU of every other layer; the data-mapper ORMs are low on both axes. CPU is the server process only; database-side CPU is excluded for all layers alike.

Same SQL, different wire protocols. Server-side capture confirms the same-SQL collapse is not a protocol artifact (Section 4): Prisma leads the same-SQL cell on MySQL using the binary protocol, yet `mysql2-tuned`'s binary path otherwise sits within noise of plain `mysql2`'s text path, so protocol choice alone does not explain the spread.

The CPU cost behind parity. Prisma shows that an ORM need not be slow, consistent with evidence that ORM cost is highly sensitive to implementation and tuning [65]: it matches or leads the native driver on several patterns but at roughly five times the application-side CPU of every other layer on the deep fetch (Figure 1), trading CPU for latency [5], a cost a throughput-only benchmark would miss. That cost is tier-specific: the application-tier efficiency ranking inverts sharply, but combined application-plus-database CPU narrows the gap to about 1.4×, since the native driver's join pushes work onto the database tier (Section 4). The equal-CPU control confirms the trade operationally, and MikroORM barely moves with more cores, consistent with a hydration cost rather than a parallelism-starved one (Section 4).

Queueing under load and the open-loop check. The open-loop cross-run (Section 4) shows that a closed-loop benchmark alone would underesti-

mate how abruptly, not just how much, a heavyweight ORM's tail degrades beyond capacity.

Writes: an engine-bound cost under default durability. RQ2 has a clear answer on inserts, measured under each engine's *default* durability configuration (Section 3): PostgreSQL's insert throughput remains layer-dependent even with fsync on, while MySQL's is governed by an engine-level commit floor common to every layer, consistent with the per-commit double flush (redo log, then binlog) InnoDB performs on every insert [26]; relaxing durability does not change this picture (Section 4). This warrants caution: a write-throughput comparison run on a single engine invites the wrong generalization either way ("the access layer is irrelevant to writes" from MySQL alone, or "matters a great deal" from PostgreSQL alone), and an engine-only benchmark such as [48] cannot expose this interaction, since it omits the access layer entirely. Only the dual-engine design exposes it, consistent with earlier quantitative ORM work showing database technology modulates mapping-strategy cost [41]. The single-row insert is not the only write shape: a transactional multi-statement write, inserting a post and five comments in one transaction, spreads the representative layers only 3.3× (Prisma 2412 down to MikroORM 740 req/s; Supplement Table S8), close to the single-row insert and far below the widest read spread, because the transaction's single commit amortizes the per-commit flush across six inserts. The layer ordering tracks the single-row insert, except for a `pg`/Prisma swap at the top (both within a few percent), so the write conclusions extend to that transactional pattern for the representative layers measured.

Methodological lessons: a checklist for access-layer benchmarking. Reaching the figures above meant confronting four confounds that would otherwise have corrupted the comparison silently. One, the aggregation fan-out, is a *correctness* bug our automated cross-check caught (the same check also caught a driver result-shape mismatch); the other three return *correct* results, invisible to any equality check, found only by performance analysis. We regard the resulting checklist as reusable:

- **Aggregation fan-out.** A naive join between `posts` and `comments` evaluated before the `SUM` multiplies each post's view count by its number of matching comment rows, inflating the aggregate, a correctness bug invisible unless every layer's output is checked against a reference, not a performance one.
- **OFFSET vs. keyset pagination.** An early, OFFSET-based design for the range-scan endpoint collapsed on MySQL at realistic offsets, which

would have been misread as an access-layer weakness rather than what it is: a pagination-strategy cost that InnoDB pays more dearly for than PostgreSQL. Rewriting every adapter around keyset pagination (`WHERE id < cursor ORDER BY id DESC LIMIT n`) removed the confound.

- **Write-workload contamination.** The insert endpoint grows `posts` on every call; because engines and layers commit at different rates, later cells in a run would scan a differently sized table depending on run order rather than on layer identity, unless benchmark-inserted rows are deleted before every cell.
- **Query-formulation confounds.** A first attempt at the aggregation endpoint pre-aggregated *all* of `comments` before filtering to one author, so every request re-scanned the whole table regardless of which author was requested, a query-plan artifact masquerading as an access-layer effect [37], fixed by rewriting the query as a correlated subquery touching only the target author's rows.

None of these four is schema-specific; each is a generic failure mode of the genre, extending the database-benchmarking pitfalls cataloged by Fruth et al. [22] and Raasveldt et al. [45] to the access-layer-comparison sub-genre. Each is easy to introduce by accident and hard to detect (the correctness ones without an automated cross-check, the performance ones without profiling every cell), and at least one plausibly explains part of the gap between some vendor-reported and independently verified figures in this space.

Practical guidance. The practical synthesis follows from RQ1–RQ3, on the default-configuration estimand a practitioner faces, and applies to the benchmarked implementations in this configuration, workload, and host rather than to access-layer categories in general. Where a service's hot path is dominated by deep/nested fetches, the access layer is consequential, and a thin layer (the native driver, a query builder, or a query-engine-based ORM such as Prisma) is the safer default *for throughput and response-time p99*: a heavyweight data-mapper ORM concentrates its largest penalty on that traffic, so weigh a query-engine ORM's parity or lead against its application-tier CPU cost wherever CPU, not throughput, is the constrained resource. Where the workload is read-simple (point reads, keyset scans) or write-bound, especially on MySQL, whose engine floor for single-row inserts dominates layer differences under default durability, the throughput cost is small, so a full ORM's productivity and type safety, not measured here, may reasonably decide the choice. A pragmatic middle path, already forced onto every layer by our aggregation endpoint, is to keep the ORM for the bulk of the application and fall back to raw SQL or the native driver only for

endpoints that assemble large object graphs; the same-SQL control shows this recovers most of the deep fetch's cost (Section 4), and second-level caching can absorb part of an ORM's read cost where the workload tolerates staleness [8].

These are performance findings only: the access-layer decision also turns on developer productivity, type safety, correctness, maintainability, and security, none of which this study measures, and which may outweigh the throughput differences reported here. Even as performance guidance, the recommendations rest on one schema, one hardware configuration, a single-host local-database deployment measured under each engine's default durability for writes, and the pinned library versions; the same-SQL comparisons bound that cost, not a license to replace idiomatic defaults with raw SQL in practice. A production deployment adds network latency, TLS, a reverse proxy, a separate database host, connection poolers, read replicas, and larger or varied payloads, any of which can shift the balance; Section 6 sets out how far these results can generalize.

6 Threats to Validity

We organize the threats along the four standard validity categories [17], [57].

Construct validity. Our dependent variables are HTTP-level throughput (req/s) and tail latency (p99) measured at the Express endpoint, not query-execution time at the driver or database boundary — deliberate, since it captures the full request round trip a deployed service incurs, including Express routing and JSON serialization, but it means the overhead we attribute to the *access layer* may partly reflect these surrounding costs. We control for this by holding those costs near-constant: every adapter satisfies a documented *adapter contract* and funnels its rows through shared canonical constructors (a fixed field set, key order, and value typing, under a fixed `TZ=UTC`), and an automated cross-check (`bench/verify.mjs`) asserted full JSON byte equality against the native-driver baseline, on both engines, across the four non-mutating patterns before any timing run (Section 3). During development that check caught a defect: PostgreSQL's schema declared `created_at` as a timezone-naive timestamp although the column is `timestamptz`, so Drizzle's PostgreSQL responses carried instants shifted by the host's UTC offset — invisible to throughput and to any coarser equivalence check; it was fixed (`withTimezone: true`) before the campaign reported here. The fixed-payload baseline floor sits well above the fastest measured cell (Section 3), so the framework floor compresses, rather than manufactures, between-layer

differences, which are therefore attributable to how each layer reaches an identical response, not to what is returned. Writes are the one endpoint that mutates state; the reported finding uses each engine's *default*, vendor-standard durability configuration (Section 3), not one we relaxed ourselves, so the write spreads (Table 5) describe a standard deployment rather than a tuning choice; a symmetric relaxed-durability secondary confirms the cross-engine gap is not an artifact of either configuration (Section 5, Supplement Table S3).

Internal validity. A naively configured ORM can trigger N+1 queries and appear artificially slow for reasons unrelated to its inherent cost; every adapter must use its layer's documented, idiomatic eager-loading strategy on the deep-fetch endpoint (Section 3). The correctness cross-check (now the byte-level comparison above) caught two defects during development, both fixed before the measurements reported here (the checklist in Section 5). Explicit warm-up phases per cell (Section 3), set to exceed the slowest layer's cold-start stabilization, absorb JIT, pool-fill, and plan-cache effects, guarding against environmentally induced measurement bias [42]; two further controls rule out non-layer confounds: benchmark rows reset before every cell (physically rebuilt for writes), and one correlated-subquery plan for aggregation across layers (Section 3). Every layer and engine within a replicate is driven by the identical, PRNG-seeded id sequence, and a forward-versus-reversed cell-order check found no detectable history effect (Section 3) — addressing the concern that processing order could confound layer identity with run position. A residual risk is coordinated omission in the load generator, which can under-report tail latency at saturation [22], [53]; a native, coordinated-omission-corrected constant-arrival cross-run on the deep fetch (Section 3) confirms the tail ranking holds under that stricter model (Section 4; Supplement Table S6).

External validity. All measurements come from a single schema, a single dataset size, one hardware configuration, and a connection pool fixed at ten, against specific engine and library versions (PostgreSQL 18.4, MySQL 9.7.1; Section 3). The fixed pool makes this an *equal configured connection limit* comparison, not a per-layer-tuned one: it isolates the access layer from pool tuning but does not show each layer's throughput under its own best pool. A per-layer pool-size frontier on both engines (Supplement Tables S7 and S24) addresses this directly: the ordering is preserved from a pool of 1 to 50, and each layer's throughput saturates well below 50, so the fixed-pool choice does not drive the ranking. A mixed read/write workload (Supplement Table S25)

likewise preserves the read ordering. Results may not transfer to larger working sets, other pool sizes, or other engine versions, and library performance ages quickly: Prisma's measured version (5.22) embeds the in-process Rust query engine our CPU accounting characterizes, whereas the version 7 line announced in late 2025 replaces it with a pure-TypeScript client [44], so the CPU findings describe the Rust-engine generation specifically, and rerunning the harness against the new architecture is the natural next step for the instrument. The host is a virtualized, 16-vCPU, single-NUMA-node cloud instance rather than dedicated bare metal, so absolute req/s figures reflect that substrate specifically; only the *relative* ordering within an engine is intended to travel. The working set is memory-resident, so these results are a hot-cache regime maximizing the visibility of layer differences; as a workload becomes I/O-bound (larger than RAM, cold caches), every layer increasingly waits on the same storage and the spreads should compress toward 1×. The deep-fetch object graph is small (a post with on average ten comments, at most 25 in the seed); the replicated fan-out sweep (Section 4) shows the spread is a roughly fixed multiplier from 0 to 500 child rows, so the headline figure is representative of the range rather than specific to one graph size. We mitigate these limitations by pinning every version, publishing the seed and an automated environment capture, and releasing the harness for re-runs on other hardware or against newer releases. A further limitation is that the load generator and server share a host, common practice in this genre that removes network variance as a confound but can understate network-bound effects; process separation on one host is not hardware isolation, since the primary campaign did not pin application, database, and generator to disjoint cores, so scheduler and shared-cache contention between them is possible (the environment capture records governor, NUMA, and affinity). A resource-isolation check re-ran the representative deep-fetch cells with the three components pinned to disjoint core sets (`ISO_` run, replication package): the isolated and shared-host medians agree within about 2% (ratios 0.98–1.01 across pg, Prisma, and MikroORM), and the layer ordering is unchanged, so same-host contention is not driving the reported ranking. The equal-CPU control (Section 4, Supplement Table S4) shows Prisma's near-native throughput depends on spare CPU, not aggregate CPU% alone. The same caveat extends to deployment topology: the server runs as a single Node.js process, whereas production deployments typically run one worker per core (`cluster`, pm2, per-pod replicas); under N workers saturating all cores, no spare cores remain for Prisma's engine threads, so the CPU-for-latency trade is a single-process result, the multi-worker case measured separately (Supplement Table S23). A sustained-load check bounds temporal drift: holding three representative layers (native `pg`, Prisma, MikroORM) un-

der ten minutes of continuous deep-fetch load, re-run under the final harness, moves throughput by at most ±1.4% between the first and last three minutes, with the band ordering preserved in every single minute, and a 90 s run reproduces the 12 s medians within 1% (replication package), so the short measured runs are not sampling a transient; longer-horizon drift over hours to days [34] remains untested. A two-machine cross-run over a dedicated link, adding real network latency, and a multi-worker variant are planned to bound this further.

Conclusion validity. Our analysis (Section 3) reports medians with the coefficient of variation and bootstrap 95% confidence intervals as stability signals and, because the design is a randomized block, compares adjacently ranked layers with *paired* tests on their per-replicate throughput ratios (paired sign-flip permutation, cross-checked with the Wilcoxon signed-rank test; Table 7). Two bounds reinforce the ordering (Section 4): the widest spreads dwarf the run-to-run CV (at most 7.6% on PostgreSQL and 6.6% on MySQL; Supplement Tables S9 and S1), and the three-band ordering holds across the concurrency sweep at ≥ 32 connections (Supplement Figure S1). On the deep/nested fetch, six of the seven adjacent pairs separate at the permutation-resolution floor ($p = 1/20{,}001$) with the faster layer ahead in nearly every replicate and survive a Bonferroni correction across the seven post-hoc adjacent-rank comparisons; the exception, native `pg` and Prisma at the top, meets the ±5% equivalence criterion under a paired TOST procedure, so we read those two as tied rather than ranked (Section 4; Table 7). Rather than assert a specific statistical power, which would require a prospective effect-size and variance model we did not prespecify, we rest the conclusions on effect sizes and interval estimates: the separating pairs combine large paired ratios with the permutation floor and non-overlapping bootstrap intervals, above the run-to-run dispersion. Booting a fresh process per replicate prevents persistent process state from carrying across replicate runs [31], but the replicates share one host and one campaign; we therefore treat replicate index and measurement order as blocks (Section 3) rather than claiming full independence, and more replicates or longer runs would tighten the intervals further.

7 Conclusion and Future Work

This paper replaces vendor-benchmark claims with a vendor-independent, reproducible comparison of relational database access layers in Express.js, serving an identical application through a representative taxonomy (native

drivers, a query builder, and six ORMs) across PostgreSQL and MySQL, over five access patterns, reporting throughput and response-time p99 together (Section 1).

The cost of abstraction is concentrated, not uniform: sharpest for the deep/nested fetch that assembles a nested object graph application-side, and modest for point reads and aggregation issued through a raw-SQL path (Section 4). The same-SQL control shows this cost is bounded but not attributable to any single mechanism (Section 4). Read rankings are stable across the two engines, and Prisma shows that, in this configuration, an ORM need not imply large overhead, at a CPU premium the equal-CPU control now quantifies directly (Section 4). Writes are the exception: the write ranking does not transfer across engines, a caution against generalizing single-engine write results (Section 5). Development also surfaced a reusable checklist of four benchmarking pitfalls specific to this genre (Section 5).

These findings held under a concurrency sweep (1 to 256 connections) and under paired permutation and Wilcoxon tests confirming adjacent-layer separation (Section 4). Further extensions would deepen the evidence: a two-machine cross-run isolating the load generator would bound observer-interference effects beyond the open-loop check performed here; additional engines and dataset sizes would widen external validity; and a dedicated study of the N+1 penalty and of cold-start latency would sharpen the practical guidance. Beyond these findings, the durable contribution is the harness itself: a vendor-independent, dual-engine instrument with a correctness cross-check and a same-SQL contrast control, released under open licenses so that these extensions, and independent re-runs against current library versions, are straightforward, addressing the repeatability gap widely documented in computer-systems research [14].

Data availability

The complete replication package (benchmark harness, database schema and deterministic seed, all adapter implementations, the byte-level correctness cross-check, raw per-cell measurements, and the scripts that generate every table and figure) is available at `https://github.com/mmiotk/express-db-access-performance` release v1.4.1 is permanently archived at Zenodo (`https://doi.org/10.5281/zenodo.21373621`). This is the generic build; the Elsevier submission build is `ist/ist_main.tex`.

CRediT authorship contribution statement

Mateusz Miotk: Conceptualization, Methodology, Software, Investigation, Data curation, Formal analysis, Visualization, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper, and is not affiliated with, nor funded by, any of the database libraries or vendors evaluated in this study.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of generative AI and AI-assisted technologies

During the preparation of this work the author used a large language model (Claude, Anthropic) to assist with drafting and editing the manuscript and with implementing and analyzing the benchmark harness. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content. All AI-assisted code was verified: the statistical estimators carry unit tests against hand-computed values and known closed-form properties (`bench/stats.test.mjs`), every reported table cell traces to a raw per-cell record through a committed generator (`MANIFEST.md`), and the byte-level correctness cross-check independently confirms that all layers return identical responses before any timing is trusted.

References

[1] A. Arcuri and L. Briand, "A hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering," *Softw. Test. Verif. Reliab.*, vol. 24, no. 3, pp. 219–250, 2014. DOI: 10.1002/stvr.1486.

[2] J. Attala and C. Khemapatpapan, "Comparing the performance of prisma, typeorm, and sequelize on postgresql," *J. Comput. Creat. Technol.*, vol. 3, no. 2, pp. 201–213, 2025. DOI: 10.14456/jcct.2025.16. [Online]. Available: https://so13.tci-thaijo.org/index.php/jcct/article/view/2330.

[3] E. Barrett, C. F. Bolz-Tereick, R. Killick, S. Mount, and L. Tratt, "Virtual machine warmup blows hot and cold," *Proceedings of the ACM on Programming Languages*, vol. 1, no. OOPSLA, pp. 1–27, 2017. DOI: 10.1145/3133876.

[4] V. R. Basili and H. D. Rombach, "The TAME project: Towards improvement-oriented software environments," *IEEE Trans. Softw. Eng.*, vol. 14, no. 6, pp. 758–773, 1988. DOI: 10.1109/32.6156.

[5] A. Bonvoisin, C. Quinton, and R. Rouvoy, "Understanding the performance-energy tradeoffs of object-relational mapping frameworks," in *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, IEEE, 2024, pp. 626–636. DOI: 10.1109/SANER60148.2024.00069.

[6] K. X. Carmo, F. Ferreira, and E. Figueiredo, "Performance evaluation of back-end frameworks: A comparative study," in *Proceedings of the 20th Brazilian Symposium on Information Systems (SBSI)*, ACM, 2024, pp. 1–9. DOI: 10.1145/3658271.3658314.

[7] I. K. Chaniotis, K.-I. D. Kyriakou, and N. D. Tselikas, "Is Node.js a viable option for building modern web applications? a performance evaluation study," *Computing*, vol. 97, no. 10, pp. 1023–1044, 2015. DOI: 10.1007/s00607-014-0394-9.

[8] T.-H. Chen, W. Shang, A. E. Hassan, M. Nasser, and P. Flora, "CacheOptimizer: Helping developers configure caching frameworks for Hibernate-based database-centric web applications," in *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, ACM, 2016, pp. 666–677. DOI: 10.1145/2950290.2950303.

[9] T.-H. Chen, W. Shang, Z. M. Jiang, A. E. Hassan, M. Nasser, and P. Flora, "Detecting performance anti-patterns for applications developed using object-relational mapping," in *Proceedings of the 36th International Conference on Software Engineering (ICSE)*, ACM, 2014, pp. 1001–1012. DOI: 10.1145/2568225.2568259.

[10] T.-H. Chen, W. Shang, Z. M. Jiang, A. E. Hassan, M. Nasser, and P. Flora, "Finding and evaluating the performance impact of redundant data access for applications that are developed using object-relational mapping frameworks," *IEEE Trans. Softw. Eng.*, vol. 42, no. 12, pp. 1148–1161, 2016. DOI: 10.1109/TSE.2016.2553039.

[11] A. Cheung, S. Madden, and A. Solar-Lezama, "Sloth: Being lazy is a virtue (when issuing database queries)," in *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data*, ACM, 2014, pp. 931–942. DOI: 10.1145/2588555.2593672.

[12] A. Cheung, A. Solar-Lezama, and S. Madden, "Optimizing database-backed applications with query synthesis," in *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, ACM, 2013, pp. 3–14. DOI: 10.1145/2491956.2462180.

[13] N. Cliff, "Dominance statistics: Ordinal analyses to answer ordinal questions," *Psychol. Bull.*, vol. 114, no. 3, pp. 494–509, 1993. DOI: 10.1037/0033-2909.114.3.494.

[14] C. Collberg and T. A. Proebsting, "Repeatability in computer systems research," *Commun. ACM*, vol. 59, no. 3, pp. 62–69, 2016. DOI: 10.1145/2812803.

[15] D. Colley, C. Stanier, and M. Asaduzzaman, "The impact of object-relational mapping frameworks on relational query performance," in *2018 International Conference on Computing, Electronics & Communications Engineering (iCCECE)*, Java, C#, Python, PHP — deliberately excludes JavaScript/Node., IEEE, 2018, pp. 47–52. DOI: 10.1109/iccecome.2018.8659222.

[16] M. Collina *et al.*, *Autocannon: Fast http/1.1 benchmarking tool for node.js*, https://github.com/mcollina/autocannon, Version 7.15.0; accessed 10 July 2026., 2023.

[17] T. D. Cook and D. T. Campbell, *Quasi-Experimentation: Design and Analysis Issues for Field Settings*. Boston, MA: Houghton Mifflin, 1979, ISBN: 9780395307908.

[18] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with YCSB," in *Proceedings of the 1st ACM Symposium on Cloud Computing (SoCC)*, ACM, 2010, pp. 143–154. DOI: 10.1145/1807128.1807152.

[19] J. Dean and L. A. Barroso, "The tail at scale," *Commun. ACM*, vol. 56, no. 2, pp. 74–80, 2013. DOI: 10.1145/2408776.2408794.

[20] Drizzle Team, *Drizzle orm benchmarks*, https://orm.drizzle.team/benchmarks, k6, 1M prepared requests, two-machine setup; reports req/s, avg + P95 latency, CPU (accessed 10 July 2026)., 2024.

[21] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA: Addison-Wesley, 2002, ISBN: 978-0-321-12742-6.

[22] M. Fruth, S. Scherzinger, W. Mauerer, and R. Ramsauer, "Tell-tale tail latencies: Pitfalls and perils in database benchmarking," in *Performance Evaluation and Benchmarking (TPCTC 2021)*, ser. Lecture Notes in Computer Science, vol. 13169, Springer, 2022, pp. 119–134. DOI: 10.1007/978-3-030-94437-7_8.

[23] Gel Data (EdgeDB), *Imdbench: Orm benchmarks*, https://github.com/geldata/imdbench, Accessed 10 July 2026., 2024.

[24] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*, ACM, 2007, pp. 57–76. DOI: 10.1145/1297027.1297033.

[25] K. Gupta and M. Mathuria, "Improving performance of web application approaches using connection pooling," in *2017 International Conference of Electronics, Communication and Aerospace Technology (ICECA)*, IEEE, 2017, pp. 355–358. DOI: 10.1109/ICECA.2017.8212833.

[26] S. Harizopoulos, D. J. Abadi, S. Madden, and M. Stonebraker, "OLTP through the looking glass, and what we found there," in *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*, ACM, 2008, pp. 981–992. DOI: 10.1145/1376616.1376713.

[27] K. Huppler, "The art of building a good benchmark," in *Performance Evaluation and Benchmarking (TPCTC 2009)*, ser. Lecture Notes in Computer Science, vol. 5895, Springer, 2009, pp. 18–30. DOI: 10.1007/978-3-642-10424-4_3.

[28] M. D. Imam Sakti, D. Dirgantara, A. K. Ramadhan, and M. A. Ibrahim, "Optimizing asynchronous performance in Node.js with Express and PostgreSQL," *Procedia Comput. Sci.*, vol. 269, pp. 172–181, 2025. DOI: 10.1016/j.procs.2025.08.270.

[29] C. Ireland, D. Bowers, M. Newton, and K. Waugh, "A classification of object-relational impedance mismatch," in *2009 First International Conference on Advances in Databases, Knowledge, and Data Applications (DBKDA)*, IEEE, 2009, pp. 36–43. DOI: 10.1109/DBKDA.2009.11.

[30] Z. M. Jiang and A. E. Hassan, "A survey on load testing of large-scale software systems," *IEEE Trans. Softw. Eng.*, vol. 41, no. 11, pp. 1091–1118, 2015. DOI: 10.1109/TSE.2015.2445340.

[31] T. Kalibera and R. Jones, "Rigorous benchmarking in reasonable time," in *Proceedings of the 2013 International Symposium on Memory Management (ISMM)*, ACM, 2013, pp. 63–74. DOI: 10.1145/2464157.2464160.

[32] B. A. Kitchenham, S. L. Pfleeger, L. M. Pickard, *et al.*, "Preliminary guidelines for empirical research in software engineering," *IEEE Trans. Softw. Eng.*, vol. 28, no. 8, pp. 721–734, 2002. DOI: 10.1109/TSE.2002.1027796.

[33] B. Krishnamachari, *How to do statistical evaluations in ECE/CS papers: A practical playbook for defensible results*, https://arxiv.org/abs/2605.00428, 2026.

[34] P. Kuffeł and B. Walter, "Changes in Node.js server-side application performance characteristics over time under load," in *Advances in Information Systems Development*, ser. Lecture Notes in Information Systems and Organisation, vol. 77, Springer, 2025, pp. 275–294. DOI: 10.1007/978-3-031-87880-0_14.

[35] C. Laaber, J. Scheuner, and P. Leitner, "Software microbenchmarking in the cloud. how bad is it really?" *Empir. Softw. Eng.*, vol. 24, no. 4, pp. 2469–2508, 2019. DOI: 10.1007/s10664-019-09681-1.

[36] R. Laigner, Y. Zhou, M. A. Vaz Salles, Y. Liu, and M. Kalinowski, "Data management in microservices: State of the practice, challenges, and research directions," *Proc. VLDB Endow.*, vol. 14, no. 13, pp. 3348–3361, 2021. DOI: 10.14778/3484224.3484232.

[37] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann, "How good are query optimizers, really?" *Proc. VLDB Endow.*, vol. 9, no. 3, pp. 204–215, 2015. DOI: 10.14778/2850583.2850594.

[38] P. Leitner and C.-P. Bezemer, "An exploratory study of the state of practice of performance testing in Java-based open source projects," in *Proceedings of the 8th ACM/SPEC International Conference on Performance Engineering (ICPE)*, ACM, 2017, pp. 373–384. DOI: 10.1145/3030207.3030213.

[39] J. Li, N. K. Sharma, D. R. K. Ports, and S. D. Gribble, "Tales of the tail: Hardware, OS, and application-level sources of tail latency," in *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*, ACM, 2014, pp. 1–14. DOI: 10.1145/2670979.2670988.

[40] Y. Li and S. Manoharan, "A performance comparison of SQL and NoSQL databases," in *2013 IEEE Pacific Rim Conference on Communications, Computers and Signal Processing (PACRIM)*, IEEE, 2013, pp. 15–19. DOI: 10.1109/PACRIM.2013.6625441.

[41] M. Lorenz, J.-P. Rudolph, G. Hesse, M. Uflacker, and H. Plattner, "Object-relational mapping revisited—a quantitative study on the impact of database technology on O/R mapping strategies," in *Proceedings of the 50th Hawaii International Conference on System Sciences (HICSS)*, 2017. DOI: 10.24251/HICSS.2017.592.

[42] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong!" In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, ACM, 2009, pp. 265–276. DOI: 10.1145/1508244.1508275.

[43] Prisma, *Query performance benchmarks*, https://benchmarks.prisma.io/, Prisma/Drizzle/TypeORM; median of 500 iterations; no throughput, no p95/p99 (accessed 10 July 2026)., 2024.

[44] Prisma, *Prisma 7 performance and benchmarks*, https://www.prisma.io/blog/prisma-7-performance-benchmarks, Announces the query-engine rewrite from the Rust engine to a TypeScript client (accessed 13 July 2026)., 2025.

[45] M. Raasveldt, P. Holanda, T. Gubner, and H. Mühleisen, "Fair benchmarking considered difficult: Common pitfalls in database performance testing," in *Proceedings of the Workshop on Testing Database Systems (DBTest)*, ACM, 2018, pp. 1–6. DOI: 10.1145/3209950.3209955.

[46] C. Russell, "Bridging the object-relational divide," *ACM Queue*, vol. 6, no. 3, pp. 18–28, 2008. DOI: 10.1145/1394127.1394139.

[47] P. J. Sadalage and M. Fowler, *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Upper Saddle River, NJ: Addison-Wesley, 2012, ISBN: 9780321826626.

[48] S. V. Salunke and A. Ouda, "A performance benchmark for the PostgreSQL and MySQL databases," *Future Internet*, vol. 16, no. 10, p. 382, 2024. DOI: 10.3390/fi16100382.

[49] B. Schroeder, A. Wierman, and M. Harchol-Balter, "Open versus closed: A cautionary tale," in *Proceedings of the 3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, USENIX Association, 2006. [Online]. Available: https://www.usenix.org/legacy/event/nsdi06/tech/schroeder.html.

[50] S. Shao, Z. Qiu, X. Yu, *et al.*, "Database-access performance antipatterns in database-backed web applications," in *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, IEEE, 2020, pp. 58–69. DOI: 10.1109/ICSME46990.2020.00016.

[51] S. E. Sim, S. Easterbrook, and R. C. Holt, "Using benchmarking to advance research: A challenge to software engineering," in *Proceedings of the 25th International Conference on Software Engineering (ICSE)*, IEEE, 2003, pp. 74–83. DOI: 10.1109/ICSE.2003.1201189.

[52] H. Sun, D. Bonetta, F. Schiavio, and W. Binder, "Reasoning about the Node.js event loop using async graphs," in *2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, IEEE, 2019, pp. 61–72. DOI: 10.1109/CGO.2019.8661173.

[53] G. Tene, *How NOT to measure latency*, https://www.infoq.com/presentations/latency-response-time/, Conference presentation; origin of the term "coordinated omission" (accessed 10 July 2026)., 2015.

[54] A. Torres, R. Galante, M. S. Pimenta, and A. J. B. Martins, "Twenty years of object-relational mapping: A survey on patterns, solutions, and their implications on application design," *Inf. Softw. Technol.*, vol. 82, pp. 1–18, 2017. DOI: 10.1016/j.infsof.2016.09.009.

[55] A. Turcotte, M. D. Shah, M. W. Aldrich, and F. Tip, "DrAsync: Identifying and visualizing anti-patterns in asynchronous JavaScript," in *Proceedings of the 44th International Conference on Software Engineering (ICSE)*, ACM, 2022, pp. 774–785. DOI: 10.1145/3510003.3510097.

[56] L. M. Vaquero, L. Rodero-Merino, and R. Buyya, "Dynamically scaling applications in the cloud," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 41, no. 1, pp. 45–52, 2011. DOI: 10.1145/1925861.1925869.

[57] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*, 2nd ed. Berlin, Heidelberg: Springer, 2012. DOI: 10.1007/978-3-642-29044-2.

[58] C. Yan, A. Cheung, J. Yang, and S. Lu, "Understanding database performance inefficiencies in real-world web applications," in *Proceedings of the 2017 ACM Conference on Information and Knowledge Management (CIKM)*, ACM, 2017, pp. 1299–1308. DOI: 10.1145/3132847.3132954.

[59] J. Yang, P. Subramaniam, S. Lu, C. Yan, and A. Cheung, "How not to structure your database-backed web applications: A study of performance bugs in the wild," in *Proceedings of the 40th International Conference on Software Engineering (ICSE)*, ACM, 2018, pp. 800–810. DOI: 10.1145/3180155.3180194.

[60] J. Yang, C. Yan, C. Wan, S. Lu, and A. Cheung, "View-centric performance optimization for database-backed web applications," in *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, IEEE, 2019, pp. 994–1004. DOI: 10.1109/ICSE.2019.00104.

[61] X. Yu, G. Bezerra, A. Pavlo, S. Devadas, and M. Stonebraker, "Staring into the abyss: An evaluation of concurrency control with one thousand cores," *Proc. VLDB Endow.*, vol. 8, no. 3, pp. 209–220, 2014. DOI: 10.14778/2735508.2735511.

[62] J. C. Yusmita, R. Arya, J. M. Wijaya, K. M. Suryaningrum, and R. R. Siswanto, "Optimizing database access strategy: A performance analysis comparison of raw sql and prisma orm," *Procedia Comput. Sci.*, vol. 269, pp. 1201–1210, 2025. DOI: 10.1016/j.procs.2025.09.061.

[63] S. Zhadko-Bazilevych, "Analysis of ORM framework approaches for Node.js," *J. Comput. Sci. Inst.*, vol. 37, pp. 426–430, 2025. DOI: 10.35784/jcsi.7951.

[64] P. van Zyl, D. G. Kourie, and A. Boake, "Comparing the performance of object databases and ORM tools," in *Proceedings of the 2006 Annual Research Conference of the South African Institute of Computer Scientists and Information Technologists (SAICSIT)*, ACM, 2006, pp. 1–11. DOI: 10.1145/1216262.1216263.

[65] P. van Zyl, D. G. Kourie, L. Coetzee, and A. Boake, "The influence of optimisations on the performance of an object relational mapping tool," in *Proceedings of the 2009 Annual Research Conference of the South African Institute of Computer Scientists and Information Tech-*

nologists (SAICSIT), ACM, 2009, pp. 150–159. DOI: 10.1145/1632149.1632169.